```
FactoryBot.define do
  factory :big_project, class: Project do
    name "Big Project"
  end
end
```

In the previous factory, all the values are static and are determined when the factory file is loaded. If you want a dynamic value to be determined when an individual factory object is created, pass a block instead of a value. The block will be evaluated when each new factory is called:

```
FactoryBot.define do
  factory :project do
    name "Project Runway"
    due_date { Date.today - rand(50) }
  end
end
```

You can also refer to a previously assigned value later in the factory:

```
FactoryBot.define do
  factory :project do
    name "Project Runway"
    due_date { Date.today - rand(50) }
    slug { "#{name.downcase.gsub(" ", "_")}" }
  end
end
```

What's nice about this is that the factory will always use the value in the name attribute to calculate the slug, even if you pass the name in yourself rather than use the default value. So this factory could be used as follows:

```
it "uses factory_bot slug block" do
  project = Factorybot.create(:project, name: "Book To Write")
  expect(project.slug).to eq("book_to_write")
end
```

If you used the include FactoryBot::Syntax::Methods call alluded to previously, then you could write the first line as project = create(:project, name: "Book To Write"). In the past I preferred the explicit reminder that factory_bot is being used, but based on what I see in code these days, I've kind of lost that battle.

Inside the factory, you can call any attribute in the model that has a setter method. Unlike with fixtures, any virtual attribute in the model (such as the unencrypted password attribute of a secure user model) is fair game.